Disagreement-Aware Multi-View Stacking for Robust Malware Classification

Kim-Huu Tran[1,3], Nguyen-Huy Le-Huu[1,3], Quoc-Huy Nguyen[1,3],
Nhut-Thanh Le-Hinh[1,2,3], Minh-Quan Ho-Le[1,2,3], and Minh-Triet Tran[1,2,3]
[1]Faculty of Information Technology, University of Science, VNUHCM
[2]Software Engineering Lab, University of Science, VNUHCM
[3]Vietnam National University, Ho Chi Minh City, Vietnam
{24120059,24120060,24120063}@student.hcmus.edu.vn
{lhnthanh,hlmquan}@selab.hcmus.edu.vn, tmtriet@fit.hcmus.edu.vn

***Abstract*—Static malware family classification can use evidence from raw byte content, disassembly, and Portable Executable metadata. Each representation captures different characteristics of a malware sample. We propose a multi-view stacking framework for Microsoft BIG 2015 that represents each sample through seven static feature views and combines their predictions at the meta level. Nine base, or Level-0, XGBoost and LightGBM classifiers generate 81 out-of-fold class probabilities. We augment these probabilities with statistics describing central tendency, inter-model dispersion, and predictive entropy, resulting in 126 disagreement-aware meta-features. Two meta-level, or Level-1, classifiers then produce predictions that are combined through constrained weight optimization. On an 80/20 train-test split, the framework achieves 99.86% accuracy, 99.83% weighted F1-score, an MCC of 0.9984, and a multiclass log loss of 0.00555. When trained on the full labeled dataset, it obtains private and public leaderboard log losses of 0.00467 and 0.00805, respectively. These results indicate that explicitly representing agreement and disagreement among heterogeneous static classifiers can improve multi-view stacking within the BIG 2015 evaluation setting.**

***Index Terms*—Malware Classification, Static Analysis, Ensemble Learning, Stacking, Gradient Boosting, Multi-view learning, Windows PE.**

I. Introduction

Malware family classification assigns a malicious executable to one of several known malware families. Static analysis performs this task without executing the sample, using information extracted from artifacts such as raw bytes, disassembly, and Portable Executable (PE) structure. The Microsoft Malware Classification Challenge (BIG 2015) provides paired `.bytes` and `.asm` artifacts and remains a widely used benchmark for Windows malware family classification [1], [2]. However, the effectiveness of a static classifier depends strongly on the selected feature representation [9].

In this paper, a *view* denotes a distinct feature representation of the same malware sample, such as byte N-grams, opcode N-grams, or PE section statistics. A single-view classifier uses a single representation, whereas a multi-view framework combines evidence from multiple representations. Stacking is a two-level ensemble strategy [14]. Level-0 base classifiers are trained on individual views and generate out-of-fold (OOF) predictions, meaning each training sample is predicted by a model not trained on that sample. Level-1 meta-classifiers then learn from these predictions. We use *inter-model disagreement* to refer to variation among the class probabilities produced by different Level-0 classifiers, while predictive entropy describes the uncertainty within each classifier's probability distribution.

Existing malware stacking systems demonstrate the value of using base-model outputs as meta-level inputs [5], [8]. Our focus is whether explicit summaries of agreement and disagreement can provide additional information beyond these outputs. Output-derived uncertainty measures have been used to characterize unreliable or ambiguous predictions [12]. We therefore augment the raw OOF probabilities with per-class aggregation and dispersion statistics, as well as per-model predictive entropy. These features make agreement and disagreement directly available to the Level-1 classifiers.

The proposed framework uses established classifiers and optimization techniques. Its contribution lies in how these components are organized and evaluated rather than in introducing a new boosting or optimization algorithm. The main contributions are threefold. First, we construct a multi-view stacking pipeline that preserves view-specific information from seven visual, lexical, and structural representations extracted from paired `.bytes` and `.asm` artifacts. Second, we augment 81 raw OOF probabilities with mean, minimum, maximum, standard deviation, and entropy features, producing a 126-dimensional meta-feature representation that explicitly captures model confidence and disagreement. Third, we combine two Level-1 classifiers through simplex-constrained SLSQP weight optimization and evaluate the contribution of feature views, engineered meta-features, and stacking components through component-wise ablation and training-ratio experiments.

II. Related Work

A. Static and Multi-View Malware Representations

Microsoft BIG 2015 supports several such representations by providing paired raw-byte and disassembly artifacts [1], [2]. Nataraj *et al.* showed that binary content can be mapped to grayscale images, enabling the use of visual texture patterns for malware classification [3]. Other static methods use lexical and structural information, including byte sequences, opcodes, API calls, strings, and PE section metadata [9].

Because each representation describes a different aspect of a sample, multi-view learning seeks to exploit complementary information across representations [10]. Chen and Ren, for example, construct a fused feature set from binary and assembly information before applying conventional classifiers, including XGBoost [11]. This feature-level fusion can provide a compact joint representation, but it combines the views before model prediction. Consequently, it does not explicitly represent whether classifiers trained on different views agree or disagree on a sample.

B. Ensemble and Stacking-Based Malware Classification

Ensemble learning combines multiple classifiers to reduce dependence on a single model or feature representation [4]. Stacked generalization extends this idea by training a meta-classifier on the predictions of base classifiers [14]. In malware analysis, MalHyStack combines heterogeneous conventional classifiers through a stacked ensemble for obfuscated malware classification [5]. Jyothsna *et al.* combine stacking with feature selection for Android malware analysis [8]. These studies demonstrate the usefulness of prediction-level fusion, although their main focus is on constructing hybrid ensembles or feature-selection strategies rather than explicitly representing disagreement among base classifiers.

Class imbalance can reduce the contribution of poorly represented families during training. We therefore apply class-weighted learning at Level-0 [13] as an implementation choice rather than a methodological contribution.

C. Disagreement-Aware Meta-Learning

Conventional stacking usually supplies the raw base-model predictions directly to a meta-classifier. However, two samples with similar average probabilities may exhibit different levels of agreement among their base classifiers. Output-derived uncertainty measures can provide additional information about unreliable or ambiguous predictions. Zoppi *et al.* combine multiple uncertainty measures to identify potentially incorrect outputs of black-box classifiers [12]. Their objective is prediction monitoring rather than malware stacking.

The proposed framework connects these directions by retaining the raw out-of-fold probabilities while adding explicit summaries of consensus, dispersion, and predictive uncertainty. In particular, the mean, minimum, maximum, and standard deviation describe how Level-0 predictions vary across classifiers, while entropy describes the uncertainty of each classifier's posterior distribution. The key difference from feature-level fusion [11] and conventional malware stacking [5], [8] is that our method retains view-specific predictions and provides their consensus and dispersion directly to the Level-1 classifiers.

III. Proposed Method

This section details the proposed multi-view stacking framework designed for robust static malware classification. Figure 1 summarizes the proposed three-stage pipeline.

A. Multi-View Feature Extraction

The framework extracts seven complementary static views from the paired `.bytes` and `.asm` files. Table I summarizes the feature engineering design. View 1 converts each `.bytes` file into a 32×32 pixel-density matrix, following the malware-visualization intuition of [3]. Views 2, 3, and 6 model lexical and string patterns, while Views 4 and 5 extract semantic and structural information from disassembly, specifically API/DLL calls and PE section statistics. View 7 consolidates these into structural tabular metadata derived from the paired artifacts.

To handle high-dimensional lexical data (V2, V3, and V6), we map raw sequential tokens into bounded sparse matrices using the hashing trick (*HashingVectorizer*). This approach avoids a fixed vocabulary and limits the dimensionality of the sparse feature matrices. We then apply chi-square (χ^2) selection to retain the most informative features:

$$\chi^2(X, y) = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(O_{i,j} - E_{i,j})^2}{E_{i,j}}, \tag{1}$$

where r is the number of possible states of the feature, c is the number of malware classes, and $O_{i,j}$ and $E_{i,j}$ represent the observed and expected contingency counts for feature state i and class j, respectively. We retain the highest-scoring 3,000-byte N-grams, 2,000 opcode N-grams, and 2,000 printable string hashes, effectively balancing descriptive power with computational efficiency.

TABLE I
Seven Static Feature Views Used in the Proposed Framework

View	Src.	Description	Dim.
V1	`.bytes`	Pixel density (32×32)	1,024
V2	`.bytes`	Byte N-grams	3,000
V3	`.asm`	Opcode N-grams	2,000
V4	`.asm`	API/DLL frequencies	173
V5	`.asm`	PE section statistics	27
V6	`.asm`	Printable strings	2,000
V7	Both	Tabular metadata	107

B. Level-0 Models and Out-of-Fold Generation

At Level-0, nine base classifiers are trained across the seven feature views. XGBoost [6] is used for dense or moderately sparse representations, including pixel density, tabular metadata, API/DLL features, opcode N-grams, and PE section statistics. LightGBM [7] is used for the high-dimensional byte N-gram and printable-string views. For the concatenated all-feature representation, XGBoost and LightGBM are trained separately, producing two additional base classifiers.

To mitigate class imbalance in BIG 2015, we apply class-balanced weighting during Level-0 training through estimator-specific weighting schemes [13]. Key hyperparameters, including maximum tree depth, learning rate, and feature subsampling ratios, are selected within the training folds using a validation-driven procedure to reduce the risk of overfitting.

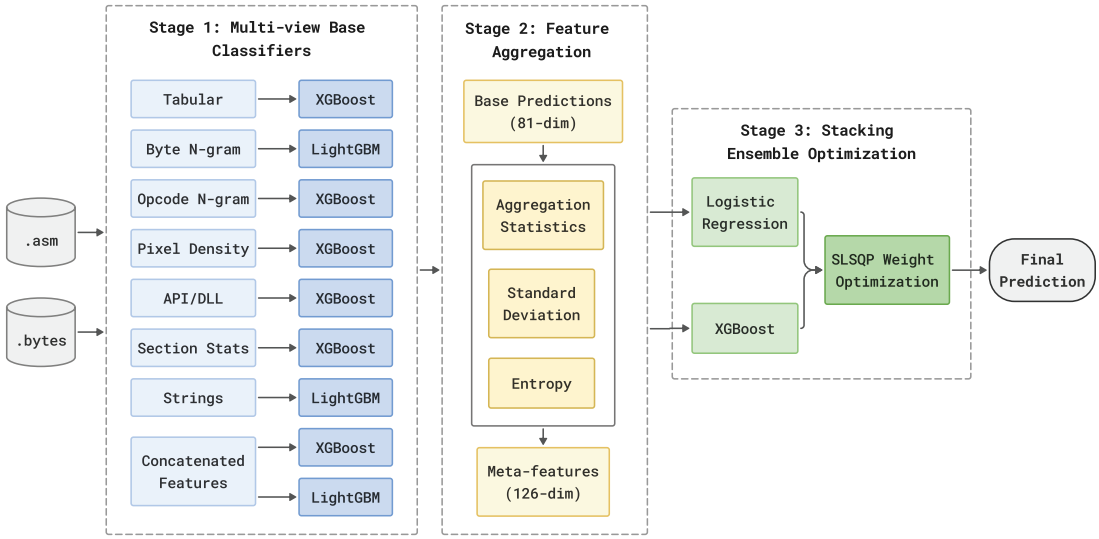

Fig. 1. Overall pipeline of the proposed multi-view stacking framework.

We employ 5-fold stratified cross-validation to generate out-of-fold (OOF) probabilities while approximately preserving the class distribution in each fold. In each iteration, the Level-0 models are trained on four folds and predict the remaining fold. Consequently, every training sample receives predictions from models that were not fitted on that sample. With nine classes and nine base classifiers, this procedure produces $9 \times 9 = 81$ OOF probability features for each sample.

C. Disagreement-Aware Meta-Feature Engineering

Let $\hat{P}_{i,c}^{(k)}$ denote the OOF probability assigned by base model $k \in \{1, \ldots, K\}$ to sample i for class $c \in \{1, \ldots, C\}$. For each sample and class, the model-aggregation statistics are

$$\mu_{i,c} = \frac{1}{K} \sum_{k=1}^{K} \hat{P}_{i,c}^{(k)}, \; \mathrm{Max}_{i,c} = \max_{k} \hat{P}_{i,c}^{(k)}, \; \mathrm{Min}_{i,c} = \min_{k} \hat{P}_{i,c}^{(k)} \tag{2}$$

and the disagreement magnitude is measured by the standard deviation

$$\sigma_{i,c} = \sqrt{\frac{1}{K} \sum_{k=1}^{K} \left(\hat{P}_{i,c}^{(k)} - \mu_{i,c} \right)^2}. \tag{3}$$

To summarize each base learner's uncertainty, we compute Shannon entropy

$$H_i^{(k)} = -\sum_{c=1}^{C} \hat{P}_{i,c}^{(k)} \log \left(\hat{P}_{i,c}^{(k)} + \epsilon \right), \tag{4}$$

where ϵ is a small constant for numerical stability.

The final meta-feature space contains 126 dimensions: 81 raw OOF probabilities, 27 aggregation features corresponding to the per-class mean, minimum, and maximum, 9 per-class standard-deviation features, and 9 per-model entropy features. The aggregation and standard-deviation features summarize consensus and dispersion across Level-0 classifiers, while entropy summarizes the uncertainty of each classifier's posterior distribution. These quantities complement rather than replace the raw OOF probabilities.

D. Level-1 Stacking and SLSQP Optimization for Ensemble Fusion

At Level-1, the 126-dimensional representation is supplied to two meta-classifiers: Logistic Regression and XGBoost. Logistic Regression models linear relationships among the Level-0 outputs and engineered statistics, whereas XGBoost can capture nonlinear interactions among these features.

The final prediction is produced by fusing the outputs of the Level-1 models. Let $\hat{q}_{i,c}^{(m)}$ denote the probability predicted by Level-1 model m for sample i and class c, and let w_m be its fusion weight. The optimal ensemble is obtained by solving a constrained optimization problem:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log \left(\sum_{m=1}^{2} w_m \hat{q}_{i,c}^{(m)} \right), \tag{5}$$

subject to:

$$\sum_{m=1}^{2} w_m = 1, \quad w_m \geq 0, \quad \forall m \in \{1, 2\}, \tag{6}$$

where N is the number of original training samples and $y_{i,c}$ is the true one-hot label indicator. We employ the Sequential Least Squares Programming (SLSQP) algorithm to minimize the multiclass log loss under these explicit simplex constraints. SLSQP selects non-negative weights that sum to one and minimize the OOF multiclass log loss.

IV. EXPERIMENTS AND RESULTS

A. Dataset

This study uses Microsoft BIG 2015 as the sole experimental benchmark. The dataset provides 10,868 labeled training samples from nine malware families together with paired `.bytes` and `.asm` files [1], [2]. Its dual representation makes

it well suited for multi-view static learning because the byte stream, disassembly structure, and derived metadata can be modeled jointly within a single stacking framework.

B. Experimental Setup

For controlled evaluation, we use a multi-stage data splitting strategy. First, the labeled dataset is partitioned into an 80% internal training set and a 20% independent test set (holdout). The 20% holdout partition is strictly reserved for the final evaluation of performance metrics and is never seen by any model during training.

Within the 80% training set, we implement a 5-fold stratified cross-validation scheme to generate the Level-0 out-of-fold (OOF) probabilities. To prevent any form of data leakage, all feature selection steps (e.g., chi-square filtering) are performed exclusively within the training folds of this 5-fold CV. The Level-1 meta-learners are then trained on the resulting 126-dimensional meta-features derived from this 80% split.

The primary evaluation uses the fixed 80%/20% split described above. Additional configurations use 20% to 90% of the labeled data for training and the remaining samples for internal evaluation. Because the holdout set varies with the training ratio, the internal metrics across these configurations are descriptive and not directly comparable. The 100% configuration uses all labeled samples and is evaluated only through the Kaggle private and public leaderboards.

C. Performance Evaluation

On the primary 80%/20% split, the final model achieves a weighted F1-score of 0.9983, an MCC of 0.9984, and an internal test log loss of 0.00555, as shown in Table II. When trained on all labeled BIG 2015 samples, the model obtains private and public leaderboard log losses of 0.00467 and 0.00805, respectively. These results show strong performance within the BIG 2015 evaluation setting, but they do not establish robustness beyond this benchmark.

The Level-1 ablations show that the two meta-classifiers make different contributions. Removing Logistic Regression increases the internal test log loss to 0.00690, while removing the XGBoost meta-classifier increases it to 0.00598. Removing SLSQP fusion increases the log loss from 0.00555 to 0.00589. Thus, the complete Level-1 configuration gives the lowest internal log loss among the evaluated variants.

D. Ablation Study

Table II presents a leave-one-out (LOO) ablation study on the 80%/20% split, quantifying the individual contributions of feature views, meta-feature subsets, and Level-1 components.

Among the feature-view ablations, removing the byte N-gram view causes the largest increase in internal test log loss, from 0.00555 to 0.01002. Removing the opcode N-gram view produces the highest private leaderboard log loss, at 0.01032. These results indicate that lexical and structural views make important but different contributions across the evaluated splits.

The *w/o Engineered Meta-features* configuration retains the 81 raw OOF probabilities while removing the 45 aggregation, standard-deviation, and entropy features. Adding these features reduces the internal test log loss from 0.00601 to 0.00555 and the private and public leaderboard log losses from 0.00840 and 0.01329 to 0.00588 and 0.00892, respectively. The aggregation statistics provide the most consistent individual contribution because their removal worsens all three results. By contrast, removing entropy or standard deviation worsens the internal and private results but improves the public score, indicating split-dependent effects. Mean, minimum, and maximum summarize the consensus statistics across classifiers, standard deviation measures inter-model dispersion, and entropy describes the uncertainty of each classifier's posterior. Thus, the results support the collective benefit of the engineered representation rather than a universal advantage for every feature group.

E. Training-Ratio Analysis

Table III reports the results for different training ratios. Because each ratio uses a different remaining holdout set, the internal metrics across rows are not directly comparable. Among the evaluated configurations, the 80% training ratio produces the lowest internal test log loss of 0.00555. The 90% configuration produces the lowest public leaderboard log loss of 0.00788.

Using all labeled samples produces the lowest private leaderboard log loss of 0.00467 and a public log loss of 0.00805. The private and public scores do not change monotonically with the training ratio. These results describe the observed configurations but do not show that increasing the training ratio always improves generalization.

F. Contextual Comparison with BIG 2015 Methods

Table IV provides a contextual comparison with representative BIG 2015 studies. The reported values are not directly comparable because the studies use different data partitions, input representations, feature-selection procedures, and metric definitions. Salas *et al.* use a 70/15/15 train–validation–test split [15], whereas Fan *et al.* use a 72/8/20 split on a labeled subset [16]. Chen and Ren report five-fold cross-validation results [11], while Khan *et al.* and our framework use separate 80/20 splits [17]. Our Level-0 OOF predictions are also generated via five-fold cross-validation on the 80% training partition.

Accordingly, Table IV should be interpreted as a summary of independently reported results rather than a controlled ranking. In particular, the log losses reported by Chen and Ren and by our framework arise from different evaluation procedures. Claims about the contribution of our method are therefore based on the controlled internal ablations in Table II, not on numerical superiority over the external studies.

G. Computational Complexity and Practical Viability

Table V reports the computational cost measured in a Kaggle environment with an NVIDIA T4 GPU and 30 GB of RAM. The complete pipeline requires approximately 39 minutes for training and occupies 49.19 MB. Inference takes 4.04 seconds for 2,174 samples, or 1.86 ms per sample. These

TABLE II
LEAVE-ONE-OUT ABLATION STUDY OF THE PROPOSED MULTI-VIEW STACKING FRAMEWORK ON THE 80%/20% BIG 2015 SPLIT

Stage	Ablated Component	Acc.	W-Prec.	W-Rec.	W-F1	MCC	Test LL	Priv. LL	Pub. LL
Stage 1	w/o XGBoost Tabular	0.9986	0.9976	0.9985	0.9980	0.9983	0.00611	0.00620	0.00702
Stage 1	w/o LGB Byte N-gram	0.9972	0.9963	0.9962	0.9962	0.9967	0.01002	0.00699	0.01096
Stage 1	w/o XGBoost Opcode	0.9972	0.9963	0.9958	0.9960	0.9967	0.00800	0.01032	0.01082
Stage 1	w/o XGBoost Pixel Density	0.9986	0.9973	0.9985	0.9979	0.9983	0.00599	0.00585	0.00970
Stage 1	w/o XGBoost API/DLL	0.9982	0.9969	0.9973	0.9971	0.9978	0.00601	0.00687	0.01005
Stage 1	w/o XGBoost PE Sections	0.9986	0.9976	0.9985	0.9980	0.9983	0.00771	0.00683	0.01111
Stage 1	w/o LGB Strings	0.9982	0.9969	0.9973	0.9971	0.9978	0.00639	0.00693	0.00821
Stage 1	w/o XGBoost All-Feature	0.9977	0.9963	0.9965	0.9964	0.9972	0.00605	0.00641	0.00823
Stage 1	w/o LGB All-Feature	0.9977	0.9966	0.9965	0.9966	0.9972	0.00743	0.00604	0.00896
Stage 2	w/o Entropy	0.9982	0.9972	0.9973	0.9972	0.9978	0.00609	0.00618	0.00806
Stage 2	w/o Aggregation Statistics	0.9982	0.9972	0.9973	0.9972	0.9978	0.00585	0.00661	0.00920
Stage 2	w/o Standard Deviation	0.9982	0.9968	0.9977	0.9972	0.9978	0.00584	0.00613	0.00776
Stage 2	w/o Engineered Meta-features	0.9986	0.9976	0.9985	0.9980	0.9983	0.00601	0.00840	0.01329
Stage 3	w/o Logistic Regression	0.9982	0.9968	0.9977	0.9972	0.9978	0.00690	0.00639	0.00747
Stage 3	w/o XGBoost Meta-model	0.9986	0.9976	0.9985	0.9980	0.9983	0.00598	0.00861	0.01303
Stage 3	w/o SLSQP Fusion	0.9982	0.9969	0.9973	0.9971	0.9978	0.00589	0.00628	0.00934
Final	**Proposed Final Model**	**0.9986**	**0.9985**	**0.9985**	**0.9983**	**0.9984**	**0.00555**	**0.00588**	**0.00892**

Acc. = accuracy, W-Prec. = weighted precision, W-Rec. = weighted recall, W-F1 = weighted F1, Test LL = test log loss, Priv. LL = private leaderboard log loss, and Pub. LL = public leaderboard log loss. Lower log-loss values are better. "w/o" denotes the removal of a specific component from the final architecture. The *w/o Engineered Meta-features* configuration retains the 81 raw OOF probability features and removes the 45 aggregation, standard-deviation, and entropy features.

TABLE III
TRAINING-RATIO ANALYSIS OF THE FINAL STACKED MODEL ON BIG 2015

Train (%)	Accuracy	W-Prec.	W-Rec.	W-F1	MCC	Test LL	Priv. LL	Pub. LL
20	0.9933	0.9842	0.9822	0.9831	0.9919	0.02971	0.01961	0.03257
30	0.9937	0.9823	0.9816	0.9819	0.9924	0.02473	0.01727	0.02440
40	0.9952	0.9832	0.9942	0.9885	0.9942	0.01883	0.00987	0.01951
50	0.9960	0.9889	0.9948	0.9917	0.9951	0.01539	0.00816	0.01802
60	0.9966	0.9832	0.9958	0.9892	0.9958	0.01477	0.00766	0.01903
70	0.9975	0.9960	0.9975	0.9968	0.9970	0.00948	0.00732	0.01247
80	0.9986	0.9985	0.9985	0.9983	0.9984	0.00555	0.00588	0.00892
90	0.9982	0.9974	0.9980	0.9977	0.9978	0.00809	0.00658	0.00788
100	-	-	-	-	-	-	0.00467	0.00805

Note: For each $x\%$ setting (from 20% to 90%), the framework is trained on $x\%$ of the labeled dataset and internally evaluated on the remaining $(100 - x)\%$ of the labeled dataset as an independent holdout set. The 100% configuration utilizes the entire labeled dataset for training; thus, it produces no internal holdout metrics (Test LL) and is evaluated exclusively via the Kaggle private and public leaderboards.

measurements are indicative and may differ in deployment environments.

V. DISCUSSION

The experiments support three main findings. First, the feature views make different contributions across the evaluated splits. Byte and opcode N-grams have the largest effects on the internal and private log losses, respectively. Second, the engineered meta-features complement the raw OOF probabilities. Their individual effects vary across splits, but the complete set improves all three reported log-loss evaluations. Third, SLSQP fusion reduces the internal test log loss from 0.00589 to 0.00555 compared with unweighted Level-1 fusion.

This study has two primary limitations. First, the evaluation is restricted to Microsoft BIG 2015. The complete pipeline requires paired `.bytes` and `.asm` artifacts because several views are extracted directly from byte dumps and disassembly. The candidate datasets available to us do not release both artifacts in a format compatible with the pipeline. Regenerating them from executable files would introduce disassembler- and configuration-dependent differences, preventing a controlled cross-dataset comparison. We therefore restrict our conclusions to BIG 2015 and leave evaluation on benchmarks with compatible paired artifacts to future work.

Second, the framework relies exclusively on static representations. Packing, encryption, and code obfuscation can suppress printable strings and alter opcode, API call, and PE section statistics [5], [9]. Byte-density and byte N-gram views may retain coarse content patterns, but the present experiments do not evaluate performance by packing status or under controlled adversarial transformations. The reported results should therefore not be interpreted as evidence of robustness to modern anti-analysis techniques. Incorporating dynamic behavior and evaluating packed or obfuscated subsets are important directions for future work.

VI. CONCLUSION AND FUTURE WORK

This paper presented a multi-view stacking framework for Windows malware family classification on the BIG 2015 dataset. The framework combines seven static feature views,

TABLE IV
CONTEXTUAL COMPARISON OF INDEPENDENTLY REPORTED RESULTS ON MICROSOFT BIG 2015. THE STUDIES USE DIFFERENT EVALUATION PROTOCOLS AND SHOULD NOT BE INTERPRETED AS A CONTROLLED RANKING.

Method	Representation / Strategy	Evaluation Protocol	Accuracy	F1-score	Log Loss
Salas et al. [15]	RGB malware images; transfer learning with MobileNet	70% train, 15% validation, 15% test	98.41%	95.96%	0.08078
Fan et al. [16]	Visualized malware images; GCSA-ResNet	72% train, 8% validation, 20% test on a labeled subset	98.58%	98.58%	–
Chen and Ren [11]	Fused binary and assembly features; XGBoost	Five-fold cross-validation on the labeled training set	99.87%	–	0.007
Khan et al. [17]	1,795 static features; D3QN sequential feature selection	80% train, 20% test	99.22%	99.20%	–
Proposed method	Seven static views; disagreement-aware stacking	80% train, 20% holdout; five-fold OOF generation within training	99.86%	99.83%	0.00555

Values are reported as provided by the corresponding studies. F1-score definitions, feature representations, data partitions, and model-selection procedures may differ. "–" denotes an unreported metric. Lower log loss is better.

TABLE V
COMPUTATIONAL COST AND EFFICIENCY PROFILE (KAGGLE ENVIRONMENT, NVIDIA T4)

Efficiency Metric	**Measurement**
Total Model Size (9 Level-0 + 2 Level-1)	49.19 MB
Total Extracted Features (across all views)	8,331
Peak Memory Usage (Resident Set Size)	5.17 GB
Level-0 Training Time (5-fold CV + Full fit)	36.8 minutes
Level-1 Meta-Learning & SLSQP Fusion Time	2.2 minutes
Total Pipeline Training Time	**∼39.0 minutes**
Inference Time (2,174 test samples)	4.04 seconds
Average Inference Latency Per Sample	**1.86 milliseconds**

nine Level-0 classifiers, 126 meta-features, two Level-1 classifiers, and constrained SLSQP fusion. On the primary 80%/20% split, it achieves an accuracy of 0.9986, a weighted F1-score of 0.9983, and a test log loss of 0.00555. When trained on all labeled samples, it obtains private and public leaderboard log losses of 0.00467 and 0.00805, respectively.

The ablation results indicate that consensus, dispersion, and entropy features complement raw OOF probabilities on BIG 2015, although individual feature groups show split-dependent effects. Future work will evaluate the framework on benchmarks that provide compatible byte-dump and disassembly artifacts, examine packed and obfuscated malware subsets, and study dynamic behavior and temporal distribution shifts.

ACKNOWLEDGMENT

This research is partially supported by the research funding from the Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam

REFERENCES

[1] R. Ronen, M. Radu, C. Feuerstein, E. Yom-Tov, and M. Ahmadi, "Microsoft Malware Classification Challenge," *arXiv:1802.10135*, 2018. doi: 10.48550/arXiv.1802.10135.

[2] Microsoft, "Microsoft Malware Classification Challenge (BIG 2015)," Kaggle, 2015. [Online]. Available: https://www.kaggle.com/c/malware-classification. Accessed: Apr. 18, 2026.

[3] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proc. 8th Int. Symp. Vis. Cyber Secur. (VizSec)*, 2011, pp. 1-7, doi: 10.1145/2016904.2016908.

[4] O. Sagi and L. Rokach, "Ensemble learning: A survey," *WIREs Data Mining Knowl. Discov.*, vol. 8, no. 4, p. e1249, 2018, doi: 10.1002/widm.1249.

[5] K. S. Roy, T. Ahmed, P. B. Udas, M. E. Karim, and S. Majumdar, "MalHyStack: A hybrid stacked ensemble learning framework with feature engineering schemes for obfuscated malware analysis," *Intell. Syst. Appl.*, vol. 20, Art. no. 200283, 2023, doi: 10.1016/j.iswa.2023.200283.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[7] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 3146-3154.

[8] V. Jyothsna, P. Mokshitha, S. Khulud, L. G. P. Reddy, N. J. Reddy, and B. Pydala, "Advancing Android security: Leveraging stacking ensemble and bioinspired feature selection for efficient malware detection," in *Proc. 5th Int. Conf. Emerg. Technol. (INCET)*, 2024, pp. 1-11, doi: 10.1109/INCET61516.2024.10593208.

[9] M. Gopinath and S. Chakkaravarthy Sethuraman, "A comprehensive survey on deep learning based malware detection techniques," *Comput. Sci. Rev.*, vol. 47, p. 100529, 2023, doi: 10.1016/j.cosrev.2022.100529.

[10] Q. Zheng, J. Zhu, Z. Li, Z. Tian, and C. Li, "Comprehensive multi-view representation learning," *Inf. Fusion*, vol. 89, pp. 198-209, Jan. 2023, doi: 10.1016/j.inffus.2022.08.014.

[11] Z. Chen and X. Ren, "An efficient boosting-based Windows malware family classification system using multi-features fusion," *Appl. Sci.*, vol. 13, no. 6, p. 4060, 2023, doi: 10.3390/app13064060.

[12] T. Zoppi, A. Ceccarelli, and A. Bondavalli, "Ensembling uncertainty measures to improve safety of black-box classifiers," *arXiv:2308.12065*, 2023, doi: 10.48550/arXiv.2308.12065.

[13] W. Chen *et al.*, "A survey on imbalanced learning: latest research, applications and future directions," *Artif. Intell. Rev.*, vol. 57, Art. no. 137, 2024, doi: 10.1007/s10462-024-10759-6.

[14] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241-259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

[15] M. I. Palma Salas, P. De Geus, and M. Botacin, "Enhancing Malware Family Classification in the Microsoft Challenge Dataset via Transfer Learning," in *Proc. 12th Latin-American Symp. Dependable and Secure Computing (LADC)*, 2023, pp. 156-163, doi: 10.1145/3615366.3615374.

[16] Y. Fan, K. Zhang, B. Zheng, Y. Zhou, J. Zhou, and W. Pan, "GCSA-ResNet: a deep neural network architecture for Malware detection," *Scientific Reports*, vol. 15, Art. no. 24098, 2025, doi: 10.1038/s41598-025-10561-6.

[17] N. Khan, A. Y. Al-Tamimi, A. Bermak, and I. M. Khalil, "Adaptive malware detection using sequential feature selection: A dueling double deep Q-Network framework for intelligent classification," *Journal of Information Security and Applications*, vol. 99, Art. no. 104407, 2026, doi: 10.1016/j.jisa.2026.104407.